

What is cultural data?

Social and Cultural Analytics (7AAVBCS5)

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

2026-10-05



Lecture Plan

Part I: Substantive overview

- Quantifying culture and its implications

Part II: Group work

- Describing data

Part III: Live coding

- Lists and arrays (numbers, text, True/False)
- Indexing and slicing
- Dictionaries
- List repetition with `*`, and f-strings
- Basic descriptive stats
 - Min, max, sum, mean, median, standard deviation
- Comparisons and `if`

This week we continue to focus on the basics of coding, but begin to think about how coding can be

used as an approach to cultural data analysis.

This will involve substantive reflections on what cultural data is, what data looks like, and performing useful operations on **lists and arrays**.

You will be introduced to some of the key ideas involved in coding for research purposes, and begin with good practices that will stand you in good stead in your academic work.

Quantifying Culture

Quantifying culture

“In general terms, quantification entails an act of translation: it expresses phenomena, characteristics or states of affairs in a general, abstract and universally accessible language – that of mathematics. This can be done by measurement or by transforming qualitative judgements, insights and observations into numeric values. Quantifications reduces a complex and confusing world to the standardized language of numbers...”

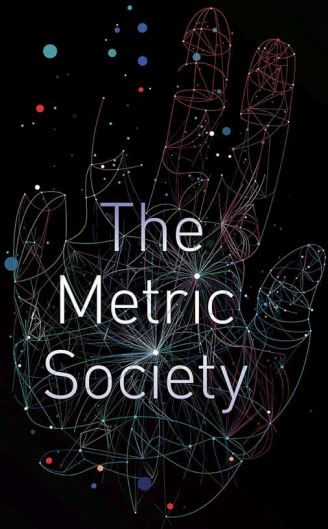
(Mau 2019, p. 13)

But what does it mean to translate culture and society into numbers?

Steffen Mau

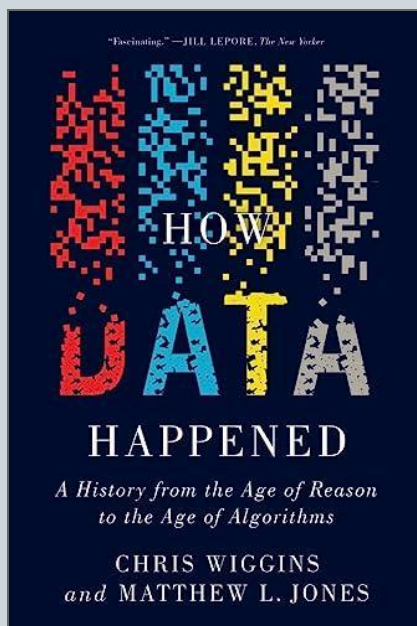
The Metric Society

On the Quantification of the Social



From experts to data

- Quantifying culture emerges in the 19th century following the Enlightenment project
- Position that (objective) data can be trusted more than (subjective) humans
- Used to challenge authority and support human progress
- “Numerical accountability, with its promise of transparency and rule-bound objectivity [...] gained prominence in conditions of distrust about experts.”
(Wiggins and Jones, p. 17)



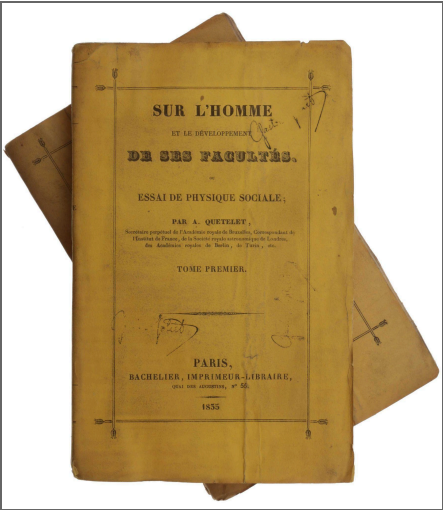
Social Physics and l'homme moyen

Developed by Belgian astronomer

Adolphe Quetelet (1796–1874)

Patterns in social phenomena can be statistically analysed to identify average population characteristics (birth of statistics in social science)

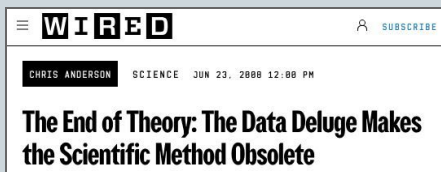
- **Crime:** stable over time and shaped by social and economic factors rather than individual morality.
- **Marriage:** followed cultural and social patterns, differing across places and classes, rather than personal decision.
- **BMI:** the idea of the “Average Man,” shaping modern notions of what is considered normal.



L'homme moyen?

- There's a risk in drawing conclusions from these quantified social traits
- “Average man” concept obscures individual diversity
- Misused in classifying people and groups (e.g., Francis Galton, eugenics, and racist theories)
- Data was seen as **laws** of society instead of **descriptions** of society

Sound familiar?



Data is made, not found

“The study of data threatened to displace other forms of expertise, from science to shop floors to the drug store. Rather than lush descriptions of countryside, a counting of flora and fauna. Rather than an ethical discussion of values, an attempt to model the effects of a given policy quantitatively. Rather than the gruesome reality of death, tables of mortality statistics. Rather than expertise about potential desires of consumers, the collection and analysis of every purchase. Rather than the clinical experience of individual physicians with a drug, randomized trials to gauge effectiveness and safety. Rather than a judgement of the character of a student applying to college, the use of standardized tests to supply an ‘objective’ measurement.”

(Wiggins and Jones, p. 22)

Numbers represent concepts

These are numerical constructs aimed at **measuring concepts**. This means there is subjectivity in datafication

- Numerical measurements are created to represent concepts
- These measurements do not represent objective reality
- Data collection reflects choices, and often societal values and biases

Data is frequently used to make claims about social reality, but there's a risk of conforming social reality to fit numerical descriptions

Always question whether measurements align with concepts you're interested in.

Critical issues in cultural data

- **Selection Bias**
 - Whose culture is recorded, preserved, digitised? (*example*)
- **Quality and Technical Challenges**
 - OCR, metadata, format obsolescence (*example*)
- **Algorithmic Bias**
 - Training data amplifies prejudices (*example*)
- **Access and Power Dynamics**
 - Paywalls, API restrictions, geographic fragmentation (*example*)
- **Indigenous Data Sovereignty**
 - FAIR vs CARE principles (*example*)

The five dimensions of context

Every dataset records some dimensions and not others. The chapter gives us a checklist:

Dimension	Unit of analysis	Typical data type in Python
Semiotic	the sign	<code>str</code> – words, tokens, labels
Temporal	the timestamp	dates, or <code>int</code> years
Spatial	the coordinate	<code>float</code> pairs, place names
Social	the agent attribute	categories – role, class, gender
Relational	the edge	pairs of things that are linked

Hill, “Contextual Analytics”, §3 and Table 1

Interrogating datasets

Group activity

Datasets, analysis, interpretation

Datasets

A collection of data, ready for analysis

Examples:

responses to a survey;

frequencies of all the words in a set of books; house prices in a city

Analysis

Identifying, quantifying and interpreting patterns in a dataset

... patterns that can be quantified, or measured

Interpretation

What does the pattern mean?

What does it tell us
about the world?

Data inspection

Choose a dataset from the list in:

- <https://msuintrodhfall2020.hcommons.org/datasets/>
- <https://openhumanitiesdata.metajnl.com/articles/>

Group activity – describe your data

- What does the dataset consist of?
- What format is the dataset in?
- What variables (and maybe types of variables) can you see in it?
- **Which of the five dimensions are present, and which are missing?**
- What sort of questions could you answer by analysing the dataset?

You will be asked to present back your main findings

Review

What we've learned so far...

- Python as a calculator
- Data types (`int`, `float`, `str`, `bool`)
- How to assign variables
- Manipulating/operating on variables
- Notebook basics

This week:

- Lists, arrays and basic descriptive analyses
- Dictionaries
- Comparisons and `if`

Lists and arrays in Python

Collections

Last week each variable held **one** value. This week, one variable holds **many**.

Python gives you two things where R gives you one:

	R	Python
Ordered collection, mixed types allowed	<i>(list)</i>	list
Ordered collection, all one type, does maths on all of it at once	vector	numpy array

If you have seen R before: `c(18, 21, 30)` is not a Python list. It is a numpy array.

Lists

An ordered collection of items. Created with square brackets.

```
1 ages = [18, 21, 30, 24, 22, 22]
2 ages
```

```
[18, 21, 30, 24, 22, 22]
```

Key characteristics:

- **Ordered** – items have positions
- **Mixed types allowed** – unlike an R vector
- **One-dimensional** – a single sequence of values

Position

Python counts from **zero**. R counts from one.

List:	[18,	21,	30,	24,	22]
Position:	0	1	2	3	4

```
1 ages = [18, 21, 30, 24, 22]
2 print(ages[0]) # the first item
3 print(ages[2]) # the third item
4 print(ages[-1]) # the last item
```

```
18
30
22
```

Slicing

Take a range of positions with a colon. The **start is included, the end is not**.

```
1 ages = [18, 21, 30, 24, 22]
2 print(ages[0:3])    # positions 0, 1, 2 -- not 3
3 print(ages[:2])     # from the start
4 print(ages[2:])     # to the end
```

```
[18, 21, 30]
[18, 21]
[30, 24, 22]
```

`ages[0:3]` gives you **three** items, not four. The end position is excluded.

Changing and growing a list

```
1 cities = ["London", "Manchester", "Rome"]
2 cities.append("Lima")           # add to the end
3 cities[0] = "Edinburgh"        # replace an item
4 print(cities)
5 print(len(cities))             # how many items
```

```
['Edinburgh', 'Manchester', 'Rome', 'Lima']
4
```

Why this matters

Instead of:

```
1 age1 = 18
2 age2 = 21
3 age3 = 30
4 age4 = 24
5 age5 = 22
```

Use a collection:

```
1 ages = [18, 21, 30, 24, 22]
```

and then work on the whole thing at once.

- One name for many values
- Operations can apply to all of it
- The foundation of the data frames we meet in Week 3

The trap: lists do not do maths

```
1 ages = [18, 21, 30]
2 ages * 2
```

```
[18, 21, 30, 18, 21, 30]
```

In R, `ages * 2` doubles every value. In Python, `ages * 2` **repeats the list**. No error.

Arrays do do maths

```
1 import numpy as np
2
3 temperatures_c = np.array([20, 25, 18, 30])
4 temperatures_f = (temperatures_c * 9 / 5) + 32
5 print(temperatures_f)
6
7 hot_days = temperatures_c > 25
8 print(hot_days)
```

```
[68.  77.  64.4 86. ]
[False False False  True]
```

Operations apply **element-wise**, automatically. This is what R's vectors do, and it is much faster than looping.

Arrays force one type

```
1 import numpy as np
2 np.array([1, 2, True, "word"])
```

```
array(['1', '2', 'True', 'word'], dtype='<U21')
```

Everything became text. Arrays **coerce** to a single type; the order is `text > number > True/False`.

```
1 print(np.array([True, False, 1])) # True/False become 1 and 0
```

```
[1 0 1]
```

Lists vs arrays

Feature	List	Array
Element types	Mixed allowed	All the same (coerced)
Structure	Flat, one-dimensional	Flat, or multi-dimensional
Creation	<code>[1, 2, 3]</code>	<code>np.array([1, 2, 3])</code>
<code>* 2</code> does	repeats the list	doubles every value
Use	general-purpose collections	data analysis, calculations

Dictionaries

A list looks things up by **position**. A dictionary looks them up by **name**.

```
1 student = {"name": "Sarah", "age": 21, "enrolled": True}
2 print(student["name"])
3 student["courses"] = ["History", "Stats"]
4 print(student.keys())
```

```
Sarah
dict_keys(['name', 'age', 'enrolled', 'courses'])
```

Describing data

```
1 import numpy as np
2
3 ages = np.array([18, 21, 30, 24, 22, 22])
4 print("count  ", len(ages))
5 print("min    ", ages.min())
6 print("max    ", ages.max())
7 print("sum    ", ages.sum())
8 print("mean   ", ages.mean())
9 print("median ", np.median(ages))
10 print("std    ", ages.std())
```

```
count    6
min      18
max      30
sum      137
mean     22.833333333333332
median   22.0
std      3.670452590924207
```

Building text

```
1 year = 1945
2 print(f"WW2 ended in {year} when the Allies won.")
3
4 name = "Alice"
5 print(f"{name} has {len(name)} letters")
```

```
WW2 ended in 1945 when the Allies won.
Alice has 5 letters
```

Anything in `{curly braces}` inside an `f"..."` string gets replaced by its value.

Comparisons and `if`

Comparisons, again

From last week:

```
1 print(3 == 4)
2 print(3 != 4)
3 print(2 <= 2)
4 print("London" == "london")
```

```
False
True
True
False
```

Comparisons produce `True` or `False`. That is what makes decisions possible.

Making a decision

```
1 temperature = 20
2
3 if temperature > 10:
4     print("go for a run")
5 else:
6     print("stay at home")
```

```
go for a run
```

- The line ends with a **colon**
- The block underneath is **indented** – four spaces
- Indentation is not decoration; it is how Python knows what belongs to the `if`

More than two outcomes

```
1 mark = 65
2
3 if mark >= 70:
4     print("First")
5 elif mark >= 60:
6     print("2:1")
7 elif mark >= 50:
8     print("2:2")
9 else:
10    print("Fail")
```

```
2:1
```

`elif` is “else if”. Python checks each in order and stops at the first true one.

Combining conditions

```
1 age = 55
2
3 if age >= 50 and age <= 60:
4     print("not young but also not old")
5
6 year = 1560
7 if year >= 1400 and year <= 1780:
8     print("Early modern")
```

```
not young but also not old
Early modern
```

and, or, not – in words, not symbols.

How not to lose an afternoon

Notebooks remember things you deleted

Cells run in the order **you click them**, not top to bottom. So a variable can still exist after you delete the cell that created it – and your notebook will work for you but fail for everyone else.

Before you submit anything: Restart and Run All

If it does not survive that, it is broken.

Four errors you will see this term

Error	Means	Usually
<code>NameError</code>	I have no variable with that name	typo, or the cell that made it never ran
<code>SyntaxError</code>	I cannot even read this line	missing <code>:</code> , bracket, or quote
<code>IndentationError</code>	the spacing does not make sense	mixed tabs and spaces
<code>TypeError</code>	you tried something on the wrong type	<code>"7" + 7</code>

Read the last line first. Everything above it is machinery.

When to use what

- **List** – a collection of things, possibly of different kinds
- **Array** – measurements you want to calculate with (temperatures, ages, scores)
- **Dictionary** – things you look up by name
- Time series, survey responses, any column of the same type -> array
- Mathematical/statistical operations -> array

Key takeaways:

1. Lists: ordered, mixed types allowed, positions start at **0**
2. Arrays: one type, and maths applies to every element at once
3. `list * 2` repeats; `array * 2` doubles
4. Dictionaries: look up by name, not position
5. Slices exclude their end point

Before next week

1. Complete `w02_worksheet.ipynb` in the workshop
2. Read Hill, “Contextual Analytics”, §3 (the five dimensions)

Note

The quiz in next week’s lecture covers **this week’s** material.

Extra



Speaker notes